

ПЗ №16. Разработка простого CLI на Python с использованием argparse

Тема

Создание консольных интерфейсов командной строки (CLI) на Python с помощью модуля `argparse`. Взаимодействие с файловой системой: просмотр, создание, удаление файлов и каталогов.

Цель работы

- **Теоретическая:** Изучить принципы обработки аргументов командной строки, структуру CLI-приложений, типы аргументов (позиционные, опциональные, флаги).
- **Практическая:** Приобрести навыки создания консольных утилит с использованием `argparse`, реализации подкоманд (субпарсеров), обработки разных типов аргументов (строки, числа, флаги), выполнения операций с файловой системой.

Задачи

1. Освоить базовый синтаксис `argparse`: создание парсера, добавление аргументов, парсинг.
2. Реализовать CLI-утилиту для работы с файлами и каталогами: просмотр содержимого, создание, удаление, получение информации.
3. Научиться использовать разные типы действий: `store_true`, `store_const`, выбор из списка.
4. Реализовать подкоманды (субпарсеры) для группировки функциональности.
5. Написать собственное CLI-приложение по варианту.

Оборудование и программное обеспечение

- **Программное обеспечение:** Python 3.8+, VS Code, терминал (командная строка).
- **Оборудование:** Персональный компьютер с ОС Windows/Linux.

Краткие теоретические сведения

CLI (Command Line Interface) – программа, управляемая через текстовые команды. Для создания CLI в Python удобно использовать встроенный модуль `argparse`.

Основные элементы `argparse`:

1. **Создание парсера:** `ArgumentParser(description='...')` .
2. **Добавление позиционных аргументов:** `add_argument('arg_name')` – обязательные аргументы, задаваемые без ключей.
3. **Добавление опциональных аргументов:** `add_argument('-f', '--file', help='...')` – необязательные, могут иметь значения или быть флагами.
4. **Типы аргументов:** `type=int` , `type=str` (по умолчанию), `type=float` , можно передать функцию.
5. **Действия:** `action='store_true'` – флаг (True/False), `action='store_const'` – константа, `action='append'` – накопление значений.
6. **Выбор из ограниченного списка:** `choices=['list', 'create', 'delete']` .
7. **Субпарсеры (подкоманды):** `subparsers = parser.add_subparsers()` – позволяют реализовать команды типа `mycli create file.txt` и `mycli delete file.txt` с разными наборами аргументов.

Пример базового CLI:

```
import argparse

parser = argparse.ArgumentParser(description='Пример CLI')
parser.add_argument('filename', help='Имя файла')
parser.add_argument('-v', '--verbose', action='store_true',
                    help='Подробный вывод')
args = parser.parse_args()

if args.verbose:
    print(f'Обработка файла {args.filename}')
else:
    print(args.filename)
```

Порядок выполнения работы

1. Подготовительный этап

1. Создайте файл `lab10_cli_[Фамилия].py` .
2. В начале файла укажите свои данные.

2. Основной этап

Задание 1. Общий пример: создание утилиты `filemanager` (выполняется всеми студентами)

Разработаем CLI-утилиту, которая поддерживает следующие подкоманды:

- `filemanager list [path]` – показать содержимое каталога
- `filemanager info <path>` – показать информацию о файле/каталоге (размер, время изменения)
- `filemanager create <filename>` – создать пустой файл
- `filemanager delete <path>` – удалить файл или пустую папку
- `filemanager rename <old> <new>` – переименовать файл/каталог

Пошаговая разработка:

Шаг 1. Импортируем модули и создаём главный парсер:

```
import os
import sys
import argparse
from datetime import datetime

def main():
    parser = argparse.ArgumentParser(description='Утилита для работы с файлами и каталогами')
    subparsers = parser.add_subparsers(dest='command', help='Доступные команды', required=True)
```

Шаг 2. Добавляем подкоманду `list`:

```
# Подкоманда list - показать содержимое каталога
parser_list = subparsers.add_parser('list', help='Показать содержимое каталога')
parser_list.add_argument('path', nargs='?', default='.', help='Путь к каталогу (по умолчанию текущий)')
parser_list.add_argument('-l', '--long', action='store_true', help='Подробный формат с размерами и датами')
```

Шаг 3. Подкоманда `info` – информация о файле/папке:

```
parser_info = subparsers.add_parser('info', help='Информация о файле/каталоге')
parser_info.add_argument('path', help='Путь к файлу или каталогу')
parser_info.add_argument('-s', '--size', action='store_true', help='Только размер')
```

Шаг 4. Подкоманда `create` – создание файла:

```
parser_create = subparsers.add_parser('create', help='Создать пустой
файл')
parser_create.add_argument('filename', help='Имя создаваемого
файла')
parser_create.add_argument('-f', '--force', action='store_true',
help='Перезаписать, если существует')
```

Шаг 5. Подкоманда `delete` – удаление:

```
parser_delete = subparsers.add_parser('delete', help='Удалить файл
или пустую папку')
parser_delete.add_argument('path', help='Путь для удаления')
parser_delete.add_argument('-r', '--recursive', action='store_true',
help='Удалить каталог рекурсивно (опасно!)')
```

Шаг 6. Подкоманда `rename` – переименование:

```
parser_rename = subparsers.add_parser('rename', help='Переименовать
файл/каталог')
parser_rename.add_argument('old', help='Текущее имя')
parser_rename.add_argument('new', help='Новое имя')
```

Шаг 7. Парсинг и выполнение команд:

```
args = parser.parse_args()

if args.command == 'list':
    cmd_list(args.path, args.long)
elif args.command == 'info':
    cmd_info(args.path, args.size)
elif args.command == 'create':
    cmd_create(args.filename, args.force)
elif args.command == 'delete':
    cmd_delete(args.path, args.recursive)
elif args.command == 'rename':
    cmd_rename(args.old, args.new)
```

Шаг 8. Реализация функций:

```

def cmd_list(path, long_mode):
    try:
        items = os.listdir(path)
        if long_mode:
            for item in items:
                full = os.path.join(path, item)
                stat = os.stat(full)
                size = stat.st_size
                mtime =
datetime.fromtimestamp(stat.st_mtime).strftime('%Y-%m-%d %H:%M')
                print(f"{size:10d} {mtime} {item}")
            else:
                for item in items:
                    print(item)
    except FileNotFoundError:
        print(f"Ошибка: путь '{path}' не существует", file=sys.stderr)
        sys.exit(1)

def cmd_info(path, only_size):
    try:
        stat = os.stat(path)
        if only_size:
            print(stat.st_size)
        else:
            print(f"Путь: {os.path.abspath(path)}")
            print(f"Размер: {stat.st_size} байт")
            print(f"Время изменения:
{datetime.fromtimestamp(stat.st_mtime)}")
            print(f"Является каталогом: {os.path.isdir(path)}")
    except FileNotFoundError:
        print(f"Ошибка: '{path}' не найден", file=sys.stderr)
        sys.exit(1)

def cmd_create(filename, force):
    if os.path.exists(filename) and not force:
        print(f"Ошибка: файл '{filename}' уже существует. Используйте -f
для перезаписи.", file=sys.stderr)
        sys.exit(1)
    try:
        with open(filename, 'w'):
            pass

```

```

        print(f"Файл '{filename}' создан")
    except Exception as e:
        print(f"Ошибка создания: {e}", file=sys.stderr)
        sys.exit(1)

def cmd_delete(path, recursive):
    if not os.path.exists(path):
        print(f"Ошибка: '{path}' не существует", file=sys.stderr)
        sys.exit(1)
    try:
        if os.path.isfile(path):
            os.remove(path)
            print(f"Удалён файл '{path}'")
        elif os.path.isdir(path):
            if recursive:
                import shutil
                shutil.rmtree(path)
                print(f"Удалён каталог '{path}' рекурсивно")
            else:
                os.rmdir(path) # удаляет только пустую папку
                print(f"Удалён пустой каталог '{path}'")
    except OSError as e:
        print(f"Ошибка удаления: {e}", file=sys.stderr)
        sys.exit(1)

def cmd_rename(old, new):
    try:
        os.rename(old, new)
        print(f"Переименован '{old}' -> '{new}'")
    except FileNotFoundError:
        print(f"Ошибка: '{old}' не найден", file=sys.stderr)
        sys.exit(1)

```

Шаг 9. Завершаем `main()` и запуск:

```

if __name__ == '__main__':
    main()

```

Тестирование (выполните в терминале):

```
python lab10_cli_Иванов.py list
python lab10_cli_Иванов.py list . -l
python lab10_cli_Иванов.py info lab10_cli_Иванов.py
python lab10_cli_Иванов.py create test.txt
python lab10_cli_Иванов.py rename test.txt new.txt
python lab10_cli_Иванов.py delete new.txt
```

Задание 2. Самостоятельная разработка CLI по варианту

Каждый вариант описывает утилиту для работы с файловой системой. Необходимо реализовать полный CLI с использованием `argparse`, включая подкоманды (субпарсеры) и различные типы опций. Требования ко всем вариантам:

- Минимум 3 подкоманды.
- Использовать `action='store_true'`, `choices`, `type=int` или `type=float`.
- Обрабатывать ошибки (неверный путь, отсутствие прав и т.п.).
- Выводить справку при `-h` или `--help`.

Вариант 1. Утилита `disk_usage`

Подкоманды:

- `usage <path>` – показать общий размер и количество файлов в каталоге (рекурсивно). Опции: `-h` (human-readable – КБ, МБ), `-d DEPTH` (глубина рекурсии).
- `largest <path>` – найти топ-5 самых больших файлов в папке (рекурсивно). Опции: `-n N` (количество выводимых файлов), `--min-size SIZE` (минимальный размер в байтах).
- `clean <path>` – удалить все файлы с заданным расширением (например, `.tmp`, `.log`). Опции: `--ext EXT` (обязательный выбор: `.tmp`, `.log`, `.bak`), `--dry-run` (только показать, что будет удалено, без реального удаления).

Вариант 2. Утилита `file_finder`

Подкоманды:

- `find <path> --name PATTERN` – поиск файлов, содержащих `PATTERN` в имени. Опции: `--type {f,d}` (файл или каталог), `--case-sensitive` (по умолчанию регистронезависимо).
- `findtext <path> --text STRING` – поиск файлов, содержащих указанную строку внутри. Опции: `--ext EXT` (искать только в файлах с расширением `.txt`, `.py` и т.д.), `--max-size MB` (не обрабатывать файлы больше указанного размера).

- `replace <path> --old TEXT --new TEXT` – замена вхождения в текстовых файлах (только файлы .txt). Опции: `--dry-run` (показать изменения без сохранения), `--backup` (создать резервную копию).

Вариант 3. Утилита `batch_rename`

Подкоманды:

- `addprefix <path> --prefix PREFIX` – добавить префикс к именам всех файлов в каталоге. Опции: `--filter EXT` (работать только с файлами определённого расширения), `--dry-run`.
- `addsuffix <path> --suffix SUFFIX` – добавить суффикс перед расширением. Опции: `--filter EXT`.
- `replace <path> --old SUBSTR --new SUBSTR` – заменить часть имени файла. Опции: `--case-sensitive`, `--dry-run`.
- `number <path> --template TEMPLATE` – переименовать файлы в формат с номерами (например, `file_01.txt`), где `TEMPLATE` – строка с `{n}` для номера. Опции: `--start N`, `--padding N`.

Вариант 4. Утилита `directory_sync`

Подкоманды:

- `mirror <source> <dest>` – синхронизировать папку назначения с исходной (копирует отсутствующие файлы, удаляет лишние в `dest`). Опции: `--exclude PATTERN` (исключить файлы по шаблону), `--dry-run`.
- `backup <source> <dest>` – создать резервную копию: копирует все файлы из `source` в `dest`, но не удаляет существующие в `dest`. Опции: `--timestamp` (добавить дату в имя папки назначения).
- `diff <dir1> <dir2>` – показать различия между двумя каталогами (какие файлы только в `dir1`, только в `dir2`, какие различаются по размеру/дате). Опции: `--ignore-size` (сравнивать только имена), `--output FILE` (записать результат в файл).

Вариант 5. Утилита `log_analyzer`

Подкоманды:

- `stats <logfile>` – анализ лог-файла (формат: каждая строка с датой, уровнем INFO/ERROR/WARNING и сообщением). Вывести количество записей по уровням. Опции: `--level LEVEL` (показать только сообщения уровня `LEVEL`), `-date DATE` (фильтр по дате YYYY-MM-DD).
- `errors <logfile>` – вывести все строки с уровнем ERROR. Опции: `--limit N` (ограничить количество), `--save FILE` (сохранить в файл).
- `report <logfile>` – создать HTML-отчёт (просто текстовый файл с расширением .html) со сводкой. Опции: `--output FILE` (имя файла отчёта), `--`


```
format {text,html} .
```

Указания по реализации (общие для всех вариантов):

- Используйте модули `os` , `os.path` , `argparse` , `sys` , `shutil` (при необходимости).
- Для рекурсивного обхода каталогов можно использовать `os.walk()` .
- В функциях обязательно обрабатывайте исключения (`PermissionError`, `FileNotFoundError`) с выводом понятного сообщения и завершением с кодом 1.
- Примерная структура кода: `main()` – создание парсера и субпарсеров, затем вызов соответствующей функции.

3. Контрольный этап

1. Для своего варианта реализуйте все указанные подкоманды и опции.
2. Протестируйте работу утилиты в терминале с разными аргументами (позиционными, опциональными, флагами).
3. Убедитесь, что справка (`-h`) корректно отображается для каждой подкоманды.
4. Проверьте обработку ошибок: попробуйте передать несуществующий путь, неправильный тип аргумента и т.д.

Контрольные вопросы

1. В чём разница между позиционными и опциональными аргументами в `argparse` ?
2. Как сделать аргумент обязательным?
3. Что означает `action='store_true'` ?
4. Как ограничить возможные значения аргумента с помощью `choices` ?
5. Зачем нужны субпарсеры (`add_subparsers`)? Приведите пример.
6. Как получить доступ к значениям аргументов после парсинга?
7. Как вывести справку, если пользователь ввел неверную команду?
8. Можно ли обрабатывать аргументы, не указанные при вызове (оставшиеся)?
9. Как реализовать подкоманду с опцией `--dry-run` , которая не выполняет реальных изменений?
10. Какие ещё библиотеки для создания CLI существуют в Python (кроме `argparse`)?

Содержание отчета

1. Тема, цель и задачи работы.
2. Код программы (общий пример и реализация своего варианта) с комментариями.

3. Примеры запуска утилиты в терминале (скриншоты или текст) для всех подкоманд с разными опциями.
4. Ответы на контрольные вопросы.
5. **Выводы по работе:** что нового узнали об `argparse` , какие возможности показались наиболее полезными, какие трудности возникли при разработке CLI. Достигнута ли цель работы.